Chapter 01.04
Binary Representation of Numbers

After reading this chapter, you should be able to:

1. *convert a base-10 real number to its binary representation,*
2. *convert a binary number to an equivalent base-10 number.*

In everyday life, we use a number system with a base of 10. For example, look at the number 257.56. Each digit in 257.56 has a value of 0 through 9 and has a place value. It can be written as

$$257.76 = 2\times10^{2} + 5\times10^{1} + 7\times10^{0} + 7\times10^{-1} + 6\times10^{-2}$$

In a binary system, we have a similar system where the base is made of only two digits 0 and 1. So it is a base 2 system. A number like (1011.0011) in base-2 represents the decimal number as

$$(1011.0011)_2 = \left((1\times2^{3} + 0\times2^{2} + 1\times2^{1} + 1\times2^{0}) + (0\times2^{-1} + 0\times2^{-2} + 1\times2^{-3} + 1\times2^{-4})\right)_{10}$$
$$= 11.1875$$

in the decimal system.

To understand the binary system, we need to be able to convert binary numbers to decimal numbers and vice-versa.

We have already seen an example of how binary numbers are converted to decimal numbers. Let us see how we can convert a decimal number to a binary number. For example take the decimal number 11.1875. First, look at the integer part: 11.

1. Divide 11 by 2. This gives a quotient of 5 and a remainder of 1. Since the remainder is 1, $a_0 = 1$.
2. Divide the quotient 5 by 2. This gives a quotient of 2 and a remainder of 1. Since the remainder is 1, $a_1 = 1$.
3. Divide the quotient 2 by 2. This gives a quotient of 1 and a remainder of 0. Since the remainder is 0, $a_2 = 0$.
4. Divide the quotient 1 by 2. This gives a quotient of 0 and a remainder of 1. Since the remainder is , $a_3 = 1$.

Since the quotient now is 0, the process is stopped. The above steps are summarized in Table 1.

$(1011.0011)_2 = 1\times2^{3} + 0\times2^{2} + 1\times2^{1} + 1\times2^{0} + 0\times2^{-1} + 0\times2^{-2} + 1\times2^{-3} + 1\times2^{-4}$

$= 8 + 0 + 2 + 1 + 0 + 0 + \frac{1}{8} + \frac{1}{16}$

$= 11 + \frac{3}{16} = 11.1875$

$11 = (1011)_2$

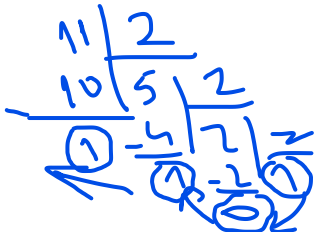


Table 1 Converting a base-10 integer to binary representation.

	Quotient	Remainder
11/2	5	$1 = a_0$
5/2	2	$1 = a_1$
2/2	1	$0 = a_2$
1/2	0	$1 = a_3$

Hence

$$(11)_{10} = (a_3 a_2 a_1 a_0)_2$$
$$= (1011)_2$$

For any integer, the algorithm for finding the binary equivalent is given in the flow chart on the next page.

Now let us look at the decimal part, that is, 0.1875.

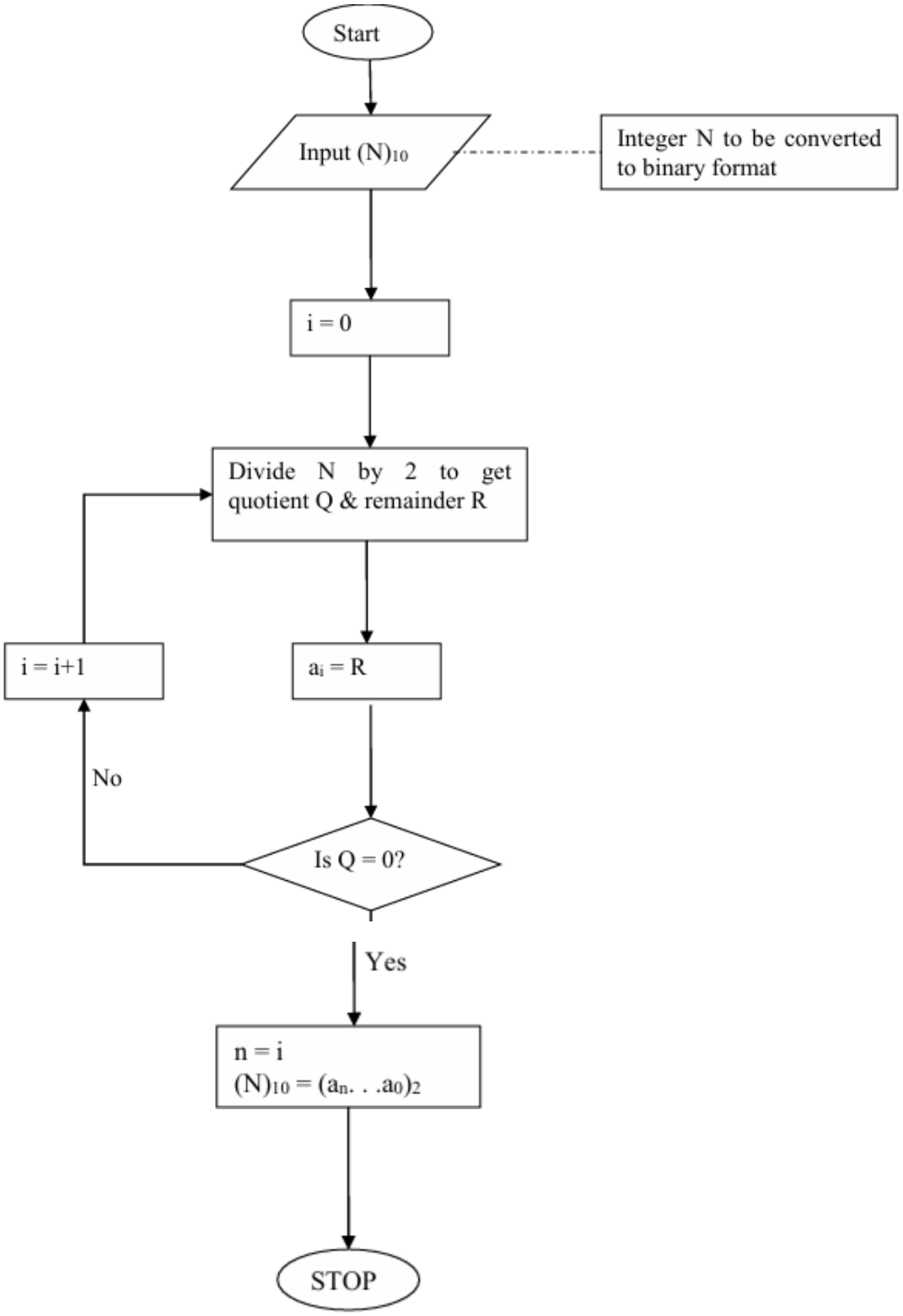
1. Multiply 0.1875 by 2. This gives 0.375. The number before the decimal is 0 and the number after the decimal is 0.375. Since the number before the decimal is 0, $a_{-1} = 0$.
2. Multiply the number after the decimal, that is, 0.375 by 2. This gives 0.75. The number before the decimal is 0 and the number after the decimal is 0.75. Since the number before the decimal is 0, $a_{-2} = 0$.
3. Multiply the number after the decimal, that is, 0.75 by 2. This gives 1.5. The number before the decimal is 1 and the number after the decimal is 0.5. Since the number before the decimal is 1, $a_{-3} = 1$.
4. Multiply the number after the decimal, that is, 0.5 by 2. This gives 1.0. The number before the decimal is 1 and the number after the decimal is 0. Since the number before the decimal is 1, $a_{-4} = 1$.

Since the number after the decimal is 0, the conversion is complete. The above steps are summarized in Table 2.

Table 2. Converting a base-10 fraction to binary representation.

	Number	Number after decimal	Number before decimal
0.1875×2	0.375	0.375	$0 = a_{-1}$
0.375×2	0.75	0.75	$0 = a_{-2}$
0.75×2	1.5	0.5	$1 = a_{-3}$
0.5×2	1.0	0.0	$1 = a_{-4}$

3


Start
Input $(N)_{10}$
Integer N to be converted to binary format
i = 0
Divide N by 2 to get quotient Q & remainder R
$a_i = R$
i = i+1
No
Is Q = 0?
Yes
n = i
$(N)_{10} = (a_n . . . a_0)_2$
STOP

Hence

$$(0.1875)_{10} = (a_{-1}a_{-2}a_{-3}a_{-4})_2$$
$$= (0.0011)_2$$

The algorithm for any fraction is given in a flowchart on the next page.
Having calculated

$$(11)_{10} = (1011)_2$$

and

$$(0.1875)_{10} = (0.0011)_2,$$

we have

$$(11.1875)_{10} = (1011.0011)_2.$$

In the above example, when we were converting the fractional part of the number, we were left with 0 after the decimal number and used that as a place to stop. In many cases, we are never left with a 0 after the decimal number. For example, finding the binary equivalent of 0.3 is summarized in Table 3.

Table 3. Converting a base-10 fraction to approximate binary representation.

	Number	Number after decimal	Number before decimal
0.3×2	0.6	0.6	$0 = a_{-1}$
0.6×2	1.2	0.2	$1 = a_{-2}$
0.2×2	0.4	0.4	$0 = a_{-3}$
0.4×2	0.8	0.8	$0 = a_{-4}$
0.8×2	1.6	0.6	$1 = a_{-5}$

Table 3. Converting a base-10 fraction to approximate binary representation.

	Number	Number after decimal	Number before decimal
0.3×2	0.6	0.6	$0 = a_{-1}$
0.6×2	1.2	0.2	$1 = a_{-2}$
0.2×2	0.4	0.4	$0 = a_{-3}$
0.4×2	0.8	0.8	$0 = a_{-4}$
0.8×2	1.6	0.6	$1 = a_{-5}$

As you can see the process will never end. In this case, the number can only be approximated in binary format, that is,

$$(0.3)_{10} \approx (a_{-1}a_{-2}a_{-3}a_{-4}a_{-5})_2 = (0.01001)_2$$

Q: But what is the mathematics behinds this process of converting a decimal number to binary format?
A: Let z be the decimal number written as

$$z = x.y$$

where

x is the integer part and y is the fractional part.

We want to find the binary equivalent of x. So we can write

$$x = a_n 2^n + a_{n-1} 2^{n-1} + \ldots + a_0 2^0$$

If we can now find $a_0, \ldots, a_n$ in the above equation then

$$(x)_{10} = (a_n a_{n-1} \ldots a_0)_2$$

We now want to find the binary equivalent of y. So we can write

$$y = b_{-1} 2^{-1} + b_{-2} 2^{-2} + \ldots + b_{-m} 2^{-m}$$

If we can now find $b_{-1}, \ldots, b_{-m}$ in the above equation then

$$(y)_{10} = (b_{-1} b_{-2} \ldots b_{-m})_2$$

Let us look at this using the same example as before.

Example 1

Convert $(11.1875)_{10}$ to base 2.

Solution

To convert $(11)_{10}$ to base 2, what is the highest power of 2 that is part of 11. That power is 3, as $2^3 = 8$ to give

$$11 = 2^3 + 3$$

What is the highest power of 2 that is part of 3. That power is 1, as $2^1 = 2$ to give

$$3 = 2^1 + 1$$

So

$$11 = 2^3 + 3 = 2^3 + 2^1 + 1$$

What is the highest power of 2 that is part of 1. That power is 0, as $2^0 = 1$ to give

$$1 = 2^0$$

Hence

$$(11)_{10} = 2^3 + 2^1 + 1 = 2^3 + 2^1 + 2^0 = 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = (1011)_2$$

To convert $(0.1875)_{10}$ to the base 2, we proceed as follows. What is the smallest negative power of 2 that is less than or equal to 0.1875. That power is -3 as $2^{-3} = 0.125$.

So

$$0.1875 = 2^{-3} + 0.0625$$

What is the next smallest negative power of 2 that is less than or equal to 0.0625. That power is -4 as $2^{-4} = 0.0625$.

So

$$0.1875 = 2^{-3} + 2^{-4}$$

Hence

$$(0.1875)_{10} = 2^{-3} + 0.0625 = 2^{-3} + 2^{-4} = 0 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3} + 1 \times 2^{-4} = (0.0011)_2$$

Since

$$(11)_{10} = (1011)_2$$

and

$$(0.1875)_{10} = (0.0011)_2$$

we get

$$(11.1875)_{10} = (1011.0011)_2$$

Can you show this algebraically for any general number?

Example 2

Convert $(13.875)_{10}$ to base 2.

Solution

For $(13)_{10}$, conversion to binary format is shown in Table 4.

Table 4. Conversion of base-10 integer to binary format.

	Quotient	Remainder
13/2	6	$1 = a_0$
6/2	3	$0 = a_1$
3/2	1	$1 = a_2$
1/2	0	$1 = a_3$

So

$$(13)_{10} = (1101)_2 .$$

Conversion of $(0.875)_{10}$ to binary format is shown in Table 5.

Table 5. Converting a base-10 fraction to binary representation.

	Number	Number after decimal	Number before decimal
0.875×2	1.75	0.75	$1 = a_{-1}$
0.75×2	1.5	0.5	$1 = a_{-2}$
0.5×2	1.0	0.0	$1 = a_{-3}$

So

$$(0.875)_{10} = (0.111)_2$$

Hence

$$(13.875)_{10} = (1101.111)_2$$

Above notes is extra material for you.

LECTURE 2

Chapter 01.05
Floating Point Representation

After reading this chapter, you should be able to:

1. *convert a base-10 number to a binary floating point representation,*
2. *convert a binary floating point number to its equivalent base-10 number,*
3. *understand the IEEE-754 specifications of a floating point representation in a typical computer,*
4. *calculate the machine epsilon of a representation.*

Consider an old time cash register that would ring any purchase between 0 and 999.99 units of money. Note that there are five (not six) working spaces in the cash register (the decimal number is shown just for clarification).

Q: How will the smallest number 0 be represented?
A: The number 0 will be represented as

0	0	0	.	0	0

Q: How will the largest number 999.99 be represented?
A: The number 999.99 will be represented as

9	9	9	.	9	9

Q: Now look at any typical number between 0 and 999.99, such as 256.78. How would it be represented?

A: The number 256.78 will be represented as

2	5	6	.	7	8

Q: What is the smallest change between consecutive numbers?
A: It is 0.01, like between the numbers 256.78 and 256.79.

Q: What amount would one pay for an item, if it costs 256.789?
A: The amount one would pay would be rounded off to 256.79 or chopped to 256.78. In either case, the maximum error in the payment would be less than 0.01.

Q: What magnitude of relative errors would occur in a transaction?
A: Relative error for representing small numbers is going to be high, while for large numbers the relative error is going to be small.

For example, for 256.786, rounding it off to 256.79 accounts for a round-off error of $256.786 - 256.79 = -0.004$. The relative error in this case is

$$\varepsilon_t = \frac{-0.004}{256.786} \times 100$$
$$= -0.001558\%.$$

For another number, 3.546, rounding it off to 3.55 accounts for the same round-off error of $3.546 - 3.55 = -0.004$. The relative error in this case is

$$\varepsilon_t = \frac{-0.004}{3.546} \times 100$$
$$= -0.11280\%.$$

Q: If I am interested in keeping relative errors of similar magnitude for the range of numbers, what alternatives do I have?

A: To keep the relative error of similar order for all numbers, one may use a floating-point representation of the number. For example, in floating-point representation, a number

256.78 is written as $+2.5678 \times 10^2$,

0.003678 is written as $+3.678 \times 10^{-3}$, and

-256.789 is written as -2.56789×10^2.

The general representation of a number in base-10 format is given as

$$sign \times mantissa \times 10^{exponent}$$

or for a number y,

$$y = \sigma \times m \times 10^e$$

Where

σ = sign of the number, +1 or -1

m = mantissa, $1 \le m < 10$

e = integer exponent (also called ficand)

scientific format in base 10

$256.78 = +2.5678 \times 10^2$

$\approx 2.568 \times 10^2$

exponent

2	5	6	8	2

Let us go back to the example where we have five spaces available for a number. Let us also limit ourselves to positive numbers with positive exponents for this example. If we use the same five spaces, then let us use four for the mantissa and the last one for the exponent. So the smallest number that can be represented is 1 but the largest number would be 9.999×10^9. By using the floating-point representation, what we lose in accuracy, we gain in the range of numbers that can be represented. For our example, the maximum number represented changed from 999.99 to 9.999×10^9.

What is the error in representing numbers in the scientific format? Take the previous example of 256.78. It would be represented as 2.568×10^2 and in the five spaces as

2	5	6	8	2

Another example, the number 576329.78 would be represented as 5.763×10^5 and in five spaces as

5	7	6	3	5

So, how much error is caused by such representation. In representing 256.78, the round off error created is $256.78 - 256.8 = -0.02$, and the relative error is

$$\varepsilon_t = \frac{-0.02}{256.78} \times 100 = -0.0077888\%,$$

In representing 576329.78, the round off error created is $576329.78 - 5.763 \times 10^5 = 29.78$, and the relative error is

$$\varepsilon_t = \frac{29.78}{576329.78} \times 100 = 0.0051672\%.$$

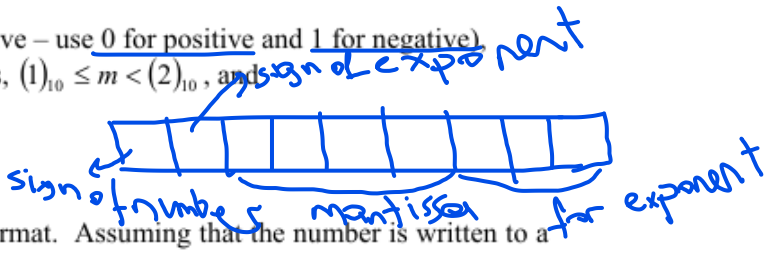
What you are seeing now is that although the errors are large for large numbers, but the relative errors are of the same order for both large and small numbers.

Q: How does this floating-point format relate to binary format?
A: A number y would be written as

$$y = \sigma \times m \times 2^e$$

Where

σ = sign of number (negative or positive – use 0 for positive and 1 for negative)
m = mantissa, $(1)_2 \le m < (10)_2$, that is, $(1)_{10} \le m < (2)_{10}$, and
e = integer exponent.



Example 1

Represent $(54.75)_{10}$ in floating point binary format. Assuming that the number is written to a hypothetical word that is 9 bits long where the first bit is used for the sign of the number, the second bit for the sign of the exponent, the next four bits for the mantissa, and the next three bits for the exponent,

Solution

$$(54.75)_{10} = (110110.11)_2 = (1.1011011)_2 \times 2^{(5)_{10}}$$

The exponent 5 is equivalent in binary format as

$$(5)_{10} = (101)_2$$

Hence

$$(54.75)_{10} = (1.1011011)_2 \times 2^{(101)_2}$$

The sign of the number is positive, so the bit for the sign of the number will have zero in it.

$$\sigma = 0$$

The sign of the exponent is positive. So the bit for the sign of the exponent will have zero in it.

The mantissa

$$m = 1011$$

(There are only 4 places for the mantissa, and the leading 1 is not stored as it is always expected to be there), and

the exponent

$$e = 101.$$

we have the representation as

0	0	1	0	1	1	1	0	1

$= +(1.1011)_2 \cdot 2^{-(110)_2} = (1 + \frac{1}{2} + \frac{1}{8} + \frac{1}{16}) \times 2^{-6}$

$= (1 + \frac{11}{16}) \times \frac{1}{64} = 0.026367$

Example 2

What number does the below given floating point format

0	1	1	0	1	1	1	1	0

represent in base-10 format. Assume a hypothetical 9-bit word, where the first bit is used for the sign of the number, second bit for the sign of the exponent, next four bits for the mantissa and next three for the exponent.

Solution

Given

Bit Representation	Part of Floating point number
0	Sign of number
1	Sign of exponent
1011	Magnitude of mantissa
110	Magnitude of exponent

The first bit is 0, so the number is positive.
The second bit is 1, so the exponent is negative.
The next four bits, 1011, are the magnitude of the mantissa, so

$$m = (1.1011)_2 = (1\times 2^0 + 1\times 2^{-1} + 0\times 2^{-2} + 1\times 2^{-3} + 1\times 2^{-4})_{10} = (1.6875)_{10}$$

The last three bits, 110, are the magnitude of the exponent, so

$$e = (110)_2 = (1\times 2^2 + 1\times 2^1 + 0\times 2^0)_{10} = (6)_{10}$$

The number in binary format then is

$$(1.1011)_2 \times 2^{-(110)_2}$$

The number in base-10 format is

$$= 1.6875 \times 2^{-6}$$
$$= 0.026367$$

Example 3

A machine stores floating-point numbers in a hypothetical 10-bit binary word. It employs the first bit for the sign of the number, the second one for the sign of the exponent, the next four for the exponent, and the last four for the magnitude of the mantissa.

a) Find how 0.02832 will be represented in the floating-point 10-bit word.
b) What is the decimal equivalent of the 10-bit word representation of part (a)?

Solution

a) For the number, we have the integer part as 0 and the fractional part as 0.02832
Let us first find the binary equivalent of the integer part

Integer part $(0)_{10} = (0)_2$

Now we find the binary equivalent of the fractional part

Fractional part: $.02832 \times 2$
0.05664×2
0.11328×2
0.22656×2

$$0.45312 \times 2$$
$$0.90624 \times 2$$
$$1.81248 \times 2$$
$$1.62496 \times 2$$
$$1.24992 \times 2$$
$$0.49984 \times 2$$
$$0.99968 \times 2$$
$$1.99936$$

Hence

$$(0.02832)_{10} \cong (0.00000111001)_2$$
$$= (1.11001)_2 \times 2^{-6}$$
$$\cong (1.1100)_2 \times 2^{-6}$$

The binary equivalent of exponent is found as follows

	Quotient	Remainder
6/2	3	$0 = a_0$
3/2	1	$1 = a_1$
1/2	0	$1 = a_2$

So

$$(6)_{10} = (110)_2$$

So

$$(0.02832)_{10} = (1.1100)_2 \times 2^{-(110)_2}$$
$$= (1.1100)_2 \times 2^{-(0110)_2}$$

Part of Floating point number	Bit Representation
Sign of number is positive	0
Sign of exponent is negative	1
Magnitude of the exponent	0110
Magnitude of mantissa	1100

The ten-bit representation bit by bit is

0	1	0	1	1	0	1	1	0	0

b) Converting the above floating point representation from part (a) to base 10 by following Example 2 gives

$$(1.1100)_2 \times 2^{-(0110)_2}$$
$$= \left(1 \times 2^0 + 1 \times 2^{-1} + 1 \times 2^{-2} + 0 \times 2^{-3} + 0 \times 2^{-4}\right) \times 2^{-\left(0 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 0 \times 2^0\right)}$$
$$= (1.75)_{10} \times 2^{-(6)_{10}}$$
$$= 0.02734375$$

Q: How do you determine the accuracy of a floating-point representation of a number?

A: The machine epsilon, $\in_{mach}$ is a measure of the accuracy of a floating point representation and is found by calculating the difference between 1 and the next number that can be represented. For example, assume a 10-bit hypothetical computer where the first bit is used for the sign of the number, the second bit for the sign of the exponent, the next four bits for the exponent and the next four for the mantissa.

We represent 1 as

0	0	0	0	0	0	0	0	0	0

and the next higher number that can be represented is

0	0	0	0	0	0	0	0	0	1

The difference between the two numbers is

$$\begin{aligned}&(1.0001)_2 \times 2^{(0000)_2} - (1.0000)_2 \times 2^{(0000)_2} \\ &= (0.0001)_2 \\ &= (1 \times 2^{-4})_{10} \\ &= (0.0625)_{10}.\end{aligned}$$

The machine epsilon is

$$\in_{mach} = 0.0625. = 2^{-4}$$

The machine epsilon, $\in_{mach}$ is also simply calculated as two to the negative power of the number of bits used for mantissa. As far as determining accuracy, machine epsilon, $\in_{mach}$ is an upper bound of the magnitude of relative error that is created by the approximate representation of a number (See Example 4).

Example 4

12

A machine stores floating-point numbers in a hypothetical 10-bit binary word. It employs the first bit for the sign of the number, the second one for the sign of the exponent, the next four for the exponent, and the last four for the magnitude of the mantissa. Confirm that the magnitude of the relative true error that results from approximate representation of 0.02832 in the 10-bit format (as found in previous example) is less than the machine epsilon.

Solution

From Example 2, the ten-bit representation of 0.02832 bit-by-bit is

0	1	0	1	1	0	1	1	0	0

Again from Example 2, converting the above floating point representation to base-10 gives

$$\begin{aligned}&(1.1100)_2 \times 2^{-(0110)_2} \\ &= (1.75)_{10} \times 2^{-(6)_{10}} \\ &= (0.02734375)_{10}\end{aligned}$$

The absolute relative true error between the number 0.02832 and its approximate representation 0.02734375 is

$$\begin{aligned}|\varepsilon_t| &= \left|\frac{0.02832 - 0.02734375}{0.02832}\right| \\ &= 0.034472\end{aligned}$$

which is less than the machine epsilon for a computer that uses 4 bits for mantissa, that is,

$$\varepsilon_{mach} = 2^{-4}$$
$$= 0.0625$$

Q: How are numbers actually represented in floating point in a real computer?
A: In an actual typical computer, a real number is stored as per the IEEE-754 (Institute of Electrical and Electronics Engineers) floating-point arithmetic format. To keep the discussion short and simple, let us point out the salient features of the single precision format.

- A single precision number uses 32 bits.
- A number y is represented as

$$y = \sigma \times (1.a_1 a_2 \cdots a_{23}) \cdot 2^e$$

where

σ = sign of the number (positive or negative)
a_i = entries of the mantissa, can be only 0 or 1, $i = 1,..,23$
e =the exponent
Note the 1 before the radix point.

- The first bit represents the sign of the number (0 for positive number and 1 for a negative number).
- The next eight bits represent the exponent. Note that there is no separate bit for the sign of the exponent. The sign of the exponent is taken care of by normalizing by adding 127 to the actual exponent. For example in the previous example, the exponent was 6. It would be stored as the binary equivalent of $127 + 6 = 133$. Why is 127 and not some other number added to the actual exponent? Because in eight bits the largest integer that can be represented is $(11111111)_2 = 255$, and halfway of 255 is 127. This allows negative and positive exponents to be represented equally. The normalized (also called biased) exponent has the range from 0 to 255, and hence the exponent e has the range of $-127 \le e \le 128$.

- If instead of using the biased exponent, let us suppose we still used eight bits for the exponent but used one bit for the sign of the exponent and seven bits for the exponent magnitude. In seven bits, the largest integer that can be represented is $(1111111)_2 = 127$ in which case the exponent e range would have been smaller, that is, $-127 \le e \le 127$. By biasing the exponent, the unnecessary representation of a negative zero and positive zero exponent (which are the same) is also avoided.
- Actually, the biased exponent range used in the IEEE-754 format is not 0 to 255, but 1 to 254. Hence, exponent e has the range of $-126 \le e \le 127$. So what are $e = -127$ and $e = 128$ used for? If $e = 128$ and all the mantissa entries are zeros, the number is $\pm\infty$ (the sign of infinity is governed by the sign bit), if $e = 128$ and the mantissa entries are not zero, the number being represented is Not a Number (NaN). Because of the leading 1 in the floating point representation, the number zero cannot be represented exactly. That is why the number zero (0) is represented by $e = -127$ and all the mantissa entries being zero.
- The next twenty-three bits are used for the mantissa.
- The largest number by magnitude that is represented by this format is

$$\left(1 \times 2^0 + 1 \times 2^{-1} + 1 \times 2^{-2} + \cdots + 1 \times 2^{-22} + 1 \times 2^{-23}\right) \times 2^{127} = 3.40 \times 10^{38}$$

The smallest number by magnitude that is represented, other than zero, is

$$\left(1\times 2^{0}+0\times 2^{-1}+0\times 2^{-2}+\cdots+0\times 2^{-22}+0\times 2^{-23}\right)\times 2^{-126}=1.18\times 10^{-38}$$

- Since 23 bits are used for the mantissa, the machine epsilon,

$$\begin{aligned}\epsilon_{mach} &= 2^{-23}\\ &= 1.19\times 10^{-7}.\end{aligned}$$

Q: How are numbers represented in floating point in double precision in a computer?
A: In double precision IEEE-754 format, a real number is stored in 64 bits.

- The first bit is used for the sign,
- the next 11 bits are used for the exponent, and
- the rest of the bits, that is 52, are used for mantissa.

Can you find in double precision the

- range of the biased exponent,
- smallest number that can be represented,
- largest number that can be represented, and
- machine epsilon?

1.5.1 Multiple-Choice Test Chapter 01.05 Floating Point Representation

1. A hypothetical computer stores real numbers in floating point format in 8-bit words. The first bit is used for the sign of the number, the second bit for the sign of the exponent, the next two bits for the magnitude of the exponent, and the next four bits for the magnitude of the mantissa. The number $e \cong 2.718$ in the 8-bit format is ()
(A) 00010101 (B) 00011010 (C) 00010011 (D) 00101010

2. A hypothetical computer stores real numbers in floating point format in 8-bit words. The first bit is used for the sign of the number, the second bit for the sign of the exponent, the next two bits for the magnitude of the exponent, and the next four bits for the magnitude of the mantissa. The number that $(10100111)_2$ represented in the above given 8-bit format is
(A) -5.75 (B) -2.875 (C) -1.75 (D) -0.359375

3. A hypothetical computer stores floating point numbers in 8-bit words. The first bit is used for the sign of the number, the second bit for the sign of the exponent, the next two bits for the magnitude of the exponent, and the next four bits for the magnitude of the mantissa. The machine epsilon is most nearly
(A) 2^{-8} (B) 2^{-4} (C) 2^{-3} (D) 2^{-2}

4. A machine stores floating point numbers in 7-bit word. The first bit is used for the sign of the number, the next three for the biased exponent and the next three for the magnitude of the mantissa. The number $(0010110)_2$ represented in base-10 is
(A) 0.375 (B) 0.875 (C) 1.5 (D) 3.5

5. A machine stores floating point numbers in 7-bit words. The first bit is stored for the sign of the number, the next three for the biased exponent and the next three for the magnitude of the mantissa. You are asked to represent 33.35 in the above word. The error you will get in this case would be
(A) underflow (B) overflow (C) NaN (D) No error will be registered.

6. A hypothetical computer stores floating point numbers in 9-bit words. The first bit is used for the sign of the number, the second bit for the sign of the exponent, the next three bits for the magnitude of the exponent, and the next four bits for the magnitude of the mantissa. Every second, the error between 0.1 and its binary representation in the 9-bit word is accumulated. The accumulated error after one day most nearly is
(A) 0.002344 (B) 20.25 (C) 202.5 (D) 8640

1.4.1 Multiple-Choice Test Chapter 01.04 Binary Representation

1. $(25)_{10} = (?)_2$
(A) 100110 (B) 10011 (C) 11001 (D) 110010

2. $(1101)_2 = (?)_{10}$
(A) 3 (B) 13 (C) 15 (D) 26

3. $(25.375)_{10} = (?.?)_2$
(A) 100110.011 (B) 11001.011 (C) 10011.0011 (D) 10011.110

4. Representing $\sqrt{2}$ in a fixed point register with 2 bits for the integer part and 3 bits for the fractional part gives a round-off error of most nearly
(A) -0.085709 (B) 0.0392 (C) 0.1642 (D) 0.2892

5. An engineer working for the Department of Defense is writing a program that transfers non-negative real numbers to integer format. To avoid overflow problems, the maximum non-negative integer that can be represented in a 5-bit integer word is
(A) 16 (B) 31 (C) 63 (D) 64

6. For a numerically controlled machine, integers need to be stored in a memory location. The minimum number of bits needed for an integer word to represent all integers between 0 and 1024 is
(A) 8 (B) 9 (C) 10 (D) 11